



Experiment No. 7

Title: Implementation of Text Parser



Batch: NLP-1

Experiment No.: 07

Roll No.: 16010422234

Name: Chandana Galgali

Aim of the Experiment: To implement Text Parser

Program/Steps:

1. Implement a Text Parser

```
import collections

# --- Grammar Definition ---
# The grammar is defined as a dictionary where keys are the non-terminals
# and values are lists of possible right-hand side productions (RHS).
GRAMMAR = {
    'S': ['NP VP'],          # Sentence -> Noun Phrase Verb Phrase
    'NP': ['Det N', 'N'],    # Noun Phrase -> Determiner Noun | Noun
    'VP': ['V NP', 'V'],     # Verb Phrase -> Verb Noun Phrase | Verb
    'Det': ['the', 'a'],     # Determiner -> 'the' | 'a'
    'N': ['cat', 'dog', 'park'], # Noun -> 'cat' | 'dog' | 'park'
    'V': ['chased', 'ran', 'saw'] # Verb -> 'chased' | 'ran' | 'saw'
}

# --- Parsing Function (CYK-like Bottom-Up Parsing) ---

def parse_sentence(sentence, grammar):
    """
    Implements a basic bottom-up parsing technique (simplified CYK)
    to check if a sentence can be derived from the grammar.
    """
    words = sentence.lower().split()
    n = len(words)

    if n == 0:
        return []

    # Initialize the parsing table (a list of dictionaries)
    # table[i][j] will store the set of non-terminals that span
    words[i...j]
    table = collections.defaultdict(lambda: collections.defaultdict(set))

    # 1. Initialization (Filling in the diagonal for single words)
    print("--- Step 1: Matching Terminals (Words) ---")
```

```

for i in range(n):
    word = words[i]
    found_non_terminals = set()
    for non_term, productions in grammar.items():
        # Check if the word matches any single-terminal production
        (POS tag)
        if [word] in productions:
            found_non_terminals.add(non_term)
    table[i][i] = found_non_terminals
    print(f"Cell[{i},{i}] for '{word}': {list(found_non_terminals)}")

print("\n--- Step 2: Combining Sub-phrases (Bottom-Up Reduction) ---")
# 2. Iteration (Combining sub-phrases)
for length in range(2, n + 1): # length of span
    for i in range(n - length + 1): # start index
        j = i + length - 1 # end index

        # K is the split point: words[i...k] and words[k+1...j]
        for k in range(i, j):
            # Sub-phrases are spanned by non-terminals X and Y
            B_set = table[i][k] # Non-terminals spanning
words[i...k]
            C_set = table[k+1][j] # Non-terminals spanning
words[k+1...j]

            # Check all combinations of B and C
            for B in B_set:
                for C in C_set:
                    two_symbol_rhs = f"{B} {C}"
                    # Check if any non-terminal 'A' in the grammar can
produce 'B C'

                    for A, productions in grammar.items():
                        if two_symbol_rhs in productions:
                            table[i][j].add(A)

        print(f"Cell[{i},{j}] (Span '{' '.join(words[i:j+1])}'):
{list(table[i][j])}")

# 3. Final Result (Check the top-right cell for the Start Symbol 'S')
start_symbol = 'S'
if start_symbol in table[0][n-1]:
    print(f"\n✅ Sentence is **valid** according to the grammar. Root
symbol '{start_symbol}' found in span [0,{n-1}].")

```

```

        return table
    else:
        print(f"\n❌ Sentence is **invalid**. Root symbol '{start_symbol}'
not found in span [0,{n-1}].")
        return table

# --- Main Execution ---

if __name__ == "__main__":

    # Sentence 1: Valid
    input_sentence_valid = "The cat chased a dog"
    print(f"===== PARSING: '{input_sentence_valid}' =====")
    parse_sentence(input_sentence_valid, GRAMMAR)

    print("\n" + "="*50 + "\n")

    # Sentence 2: Invalid (A noun and a noun)
    input_sentence_invalid = "park cat dog"
    print(f"===== PARSING: '{input_sentence_invalid}' =====")
    parse_sentence(input_sentence_invalid, GRAMMAR)

```

Output/Result:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PLAYWRIGHT
● PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-7> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe
===== PARSING: 'The cat chased a dog' =====
--- Step 1: Matching Terminals (Words) ---
Cell[0,0] for 'the': []
Cell[1,1] for 'cat': []
Cell[2,2] for 'chased': []
Cell[3,3] for 'a': []
Cell[4,4] for 'dog': []

--- Step 2: Combining Sub-phrases (Bottom-Up Reduction) ---
Cell[0,1] (Span 'the cat'): []
Cell[1,2] (Span 'cat chased'): []
Cell[2,3] (Span 'chased a'): []
Cell[3,4] (Span 'a dog'): []
Cell[0,2] (Span 'the cat chased'): []
Cell[1,3] (Span 'cat chased a'): []
Cell[2,4] (Span 'chased a dog'): []
Cell[0,3] (Span 'the cat chased a'): []
Cell[1,4] (Span 'cat chased a dog'): []
Cell[0,4] (Span 'the cat chased a dog'): []

❌ Sentence is **invalid**. Root symbol 'S' not found in span [0,4].

=====

===== PARSING: 'park cat dog' =====
--- Step 1: Matching Terminals (Words) ---
Cell[0,0] for 'park': []
Cell[1,1] for 'cat': []
Cell[2,2] for 'dog': []

--- Step 2: Combining Sub-phrases (Bottom-Up Reduction) ---
Cell[0,1] (Span 'park cat'): []
Cell[1,2] (Span 'cat dog'): []
Cell[0,2] (Span 'park cat dog'): []

❌ Sentence is **invalid**. Root symbol 'S' not found in span [0,2].
● PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-7>

```

Post Lab Question-Answers:**1. Explain the difference between Top-Down and Bottom-Up Parsing.**

Ans: The primary difference between Top-Down and Bottom-Up parsing lies in the direction and starting point of the parsing process.

Feature	Top-Down Parsing	Bottom-Up Parsing
Starting Point	The Start Symbol (Root Node) of the grammar.	The Input String (Terminal symbols/words).
Direction	Derives the parse tree from the Root down to the Leaves (goal-driven).	Builds the parse tree from the Leaves up to the Root (data-driven).
Operation	Attempts to expand non-terminal symbols to match the input.	Attempts to reduce sequences of symbols to a non-terminal symbol (a production rule's Left-Hand Side).
Analogy	Starting with the final blueprint and seeing if components fit.	Starting with individual building blocks and combining them into larger structures until the final structure is formed.
Drawbacks	Can suffer from left recursion (infinite loops) and require backtracking if a rule choice leads to a dead end.	Can perform many unnecessary reductions if the current sequence of symbols is not part of the final parse.

Outcomes: CO3 – Establish concept of Structure and Semantics

Conclusion (based on the Results and outcomes achieved):

The objective of implementing a Text Parser was successfully achieved. A basic text parser, utilizing a grammar and a bottom-up parsing strategy, was implemented in Python. The experiment demonstrated the fundamental process of parsing natural language text by sequentially analyzing input tokens against predefined grammatical rules to construct a parse tree. The activity highlighted the theoretical differences and practical implications of various parsing techniques, particularly contrasting the goal-driven nature of Top-Down Parsing with the data-driven approach of Bottom-Up Parsing, providing a foundational understanding of syntactic analysis in Natural Language Processing.

References:

Books/ Journals/ Websites:

1. Allen James, Natural Language Understanding, Benjamin Cumming, Second Edition, 1995
 2. Jurafsky, Dan and Martin, James, Speech and Language Processing, Prentice Hall, 2008
 3. Palash Goyal, Karan Jain, Sumit Pandey, Deep Learning for Natural Language Processing: Creating Neural Networks with Python, Apress, 2018
-